Welcome

to

DHT·SR04 센서 통신

2조 정준혁 김성은 이시원 황영빈

2026.11.02

1. 팀원소개
2. 개요
3. DHT 센서
4. SR04 센서
5. Uart- Ascii
6. 결과
7. 고찰

팀원 소개

팀원 간 역할

정준혁

통합 · 제어 · 검증

- 전체 모듈 통합
- 컨트롤 유닛 구현
- 전체 검증

김성은

ASCII 모듈 2종

- ASCII Decoder 구현
- ASCII Sender 구현
- 2개 모듈 검증

이시원

DHT 센서

- DHT 센서 유닛 구현
- DHT 센서 유닛 검증

황영빈

SR04 센서

- SR04 센서 유닛 구현
- SR04 센서 유닛 검증

개요

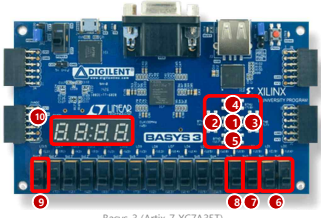
| Basys-3 기반 Watch · Stopwatch · DHT · SR04 · UART 센서 통합 및 제어 시스템 구현

◆ **목표**
기존에 구현한 Watch, Stopwatch에 이어서 DHT 센서와 SR04 초음파 센서를 추가하고 각 모드에 맞는 값을 FND로 확인.
버튼과 Comport Master를 이용한 명령어 입력으로 출력을 제어

입력 스위치 및 버튼 기능

#	보드 입력	Pin	Stopwatch (00)	Watch (01)	SR04 (10)	DHT11 (11)	UART ASCII
①	**btnC**	U18	전체 reset → Stopwatch 00:00:00.00 · Watch 12:00:00.00				-
②	**btnL**	W19	CLEAR	설정 자리 ← 이동	-	-	**c / C**
③	**btnR**	T17	RUN / STOP	설정 자리 → 이동	-	-	**r / R**
④	**btnU**	T18	UP ↔ DOWN	선택 자리 +1	-	-	**u / U**
⑤	**btnD**	U17	LAP	선택 자리 −1	-	-	**l / L**
⑥	**sw[1:0]**	V17 · V16	모드 선택 00 Stopwatch · 01 Watch · 10 SR04 · 11 DHT11 (LED[1:0] 표시, SR04·DHT11 진입 시 측정 시작)				**0 1 2 3**
⑦	**sw[2]**	W16	-	1 : 시간 설정 모드	-	-	-
⑧	**sw[3]**	W17	-	-	-	0 온도 / 1 습도	-
⑨	**sw[15]**	R2	1 : 시.분 (0 : 초.ms)	1 : 초.ms (0 : 시.분)	-	-	-
⑩	**FND**	com·seg	경과 시간	현재 시각	거리 (cm)	온도 / 습도	-

Description of I/O Used



Basys-3 (Artix-7 XC7A35T)

■ **사용 I/O** clk (W5) · btn ①~⑤ · sw ⑥~⑨ · FND ⑩

Note. 버튼 디바운스 : 100 Hz 샘플링 8회 연속 HIGH (약 80 ms) / 시간 tick : 100 Hz (10 ms) / 범위 : msec 0~99, sec·min 0~59, hour 0~23
CLEAR · MODE(UP↔DOWN) : STOP 상태에서만 / LAP : RUN 중에만 동작 (3 초 표시) / Pin : Basys-3 기본 핀 (sw[2] W16 · sw[3] W17 · sw[15] R2)

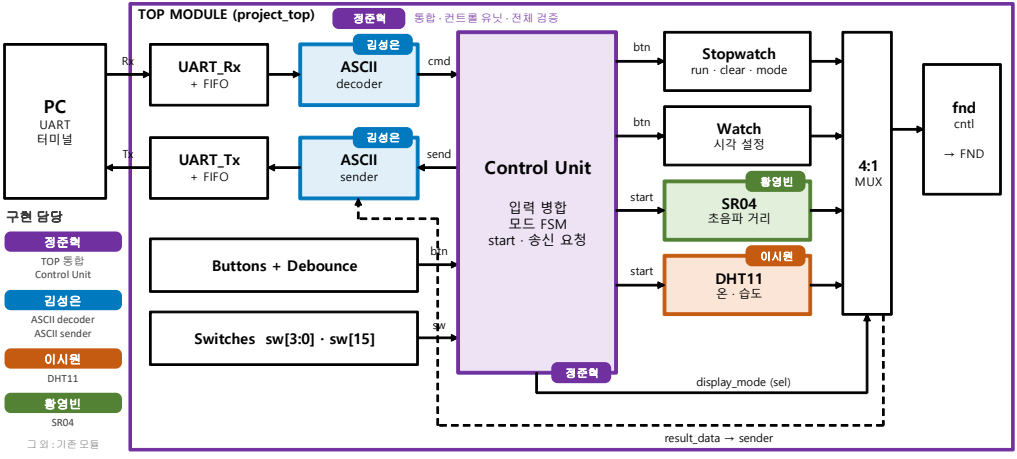
ASCII 명령어 별 동작 효과

Comport Master → UART (9600 baud) → ASCII Decoder → Control Unit : 버튼 · 스위치와 같은 동작을 PC 에서 수행

ASCII	대응 입력	Stopwatch	Watch	SR04	DHT11	PC 응답 (UART TX)
r / R	btnR	RUN ↔ STOP 전환	설정 자리 → 이동	-	-	**동작 반영 후 현재 값 송신** HH:MM:SS:MS 예) 00:00:00:05 SR04 · DHT11 모드에서는 입력 무시
c / C	btnL	CLEAR (STOP 상태)	설정 자리 ← 이동	-	-	
u / U	btnU	UP ↔ DOWN (STOP 상태)	선택 자리 +1	-	-	
l / L	btnD	LAP (RUN 중, 3초 표시)	선택 자리 −1	-	-	
0	sw[1:0] = 00	STOPWATCH 모드로 전환 (LED = 00)				HH:MM:SS:MS 예) 00:00:00:00
1	sw[1:0] = 01	WATCH 모드로 전환 (LED = 01)				HH:MM:SS:MS 예) 12:00:00:00
2	sw[1:0] = 10	SR04 모드로 전환 + 거리 측정 시작 (sr04_start)				측정 완료 시 0010 (cm) 으로 표시 timeout 시 xxxx 으로 표시
3	sw[1:0] = 11	DHT11 모드로 전환 + 온·습도 측정 시작 (dht11_start)				완료 시 H=45.00(%) T=25.30(C) 표시 checksum 오류 시 xxxx 으로 표시

Block Diagram

| 모든 입력은 Control Unit 으로 모임. display_mode로 FND 와 PC 송신값을 함께 선택



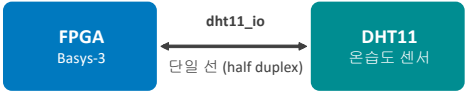
DHT 센서 개요

DHT11 : 온도·습도를 단일 선(Single Wire) Half Duplex 방식으로 보내 주는 디지털 센서

DHT11 의 역할

- 온도 0~50℃ · 습도 20~90%RH 를 측정해 디지털 값으로 출력
- FPGA 가 start 신호를 보낼 때만 응답
- 한 번의 통신 = 40 bit 전송 (센서 → FPGA), 약 4 ms 소요

FPGA ⇄ DHT11 연결



start 요청 (버튼 or 명령어) → 센서 응답 → 40 bit 데이터

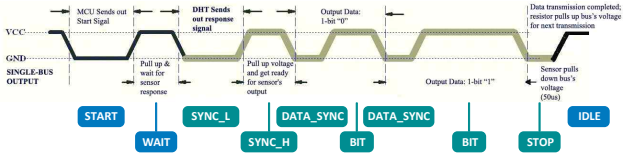
전송 데이터 : 40 bit (MSB first)

습도 정수	습도 소수	온도 정수	온도 소수	Checksum
8 bit	8 bit	8 bit	8 bit	8 bit
data[39:32]	data[31:24]	data[23:16]	data[15:8]	data[7:0]

예) 37h · 00h · 18h · 03h · 52h = 습도 55.0 % , 온도 24.3 ℃
Checksum = 앞 4 byte 합의 하위 8 bit → 일치하면 valid = 1

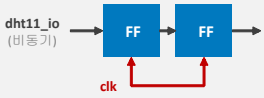
구현 포인트 : Half-Duplex 와 CDC를 고려함

한 선을 FPGA 와 센서가 번갈아 구동 & 센서 신호는 clk 와 비동기



선 : 검정 = FPGA(MCU), 회색 = DHT11 라벨 : 파랑 = FPGA 구동 · 청록 = DHT11 구동

2-FF 동기화기



io_sync[0] → io_sync[1] = io_s

1 Half-Duplex : 한 선을 번갈아 구동

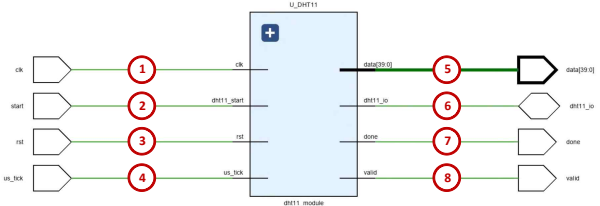
- 데이터 선 1개를 FPGA 와 DHT11 이 시간을 나눠 사용
- START·WAIT : FPGA 가 구동 (LOW 20ms → HIGH 30us)
- SYNC_L ~ STOP : FPGA 는 선을 놓고(Hi-Z) 입력만, DHT11 구동

2 CDC : 비동기 입력을 clk 에 동기화

- 센서 신호는 FPGA clk 와 무관
  → 그대로 쓰면 메타스터빌리티 위험
- FSM 은 동기화된 io_s만 읽음 (지연 수 clk ≪ 1us)

DHT 컨트롤 유닛 : 포트 구성

dht11_module — in / out 포트와 역할



No	Input	역할
1	**clk**	시스템 클록 (100 MHz) 연결
2	**dht11_start**	DHT11 동작을 시작시키는 신호 (버튼 / ASCII)
3	**rst**	리셋 신호 (버튼 / ASCII)
4	**us_tick**	us_tick 생성 모듈과 연결 (1us 펄스)

No	Output	역할
5	**data[39:0]**	송신 40 bit (습도·온도 정수/소수, checksum)
6	**dht11_io**	DHT11 과 연결하는 inout (단일 선)
7	**done**	측정 1회 완료 1clk 펄스 (data 갱신)
8	**valid**	checksum 일치 여부 (LED 연결)

DHT 컨트롤 유닛 : 검증 구성

센서 모델 Testbench · TC00 ~ TC16 (17개 시나리오) · 자동 check() 판정

CASE A TC00, TC10

Reset

리셋 시 IDLE 복귀, 측정 중 리셋 후 재측정

14 PASS / 0 FAIL

CASE B TC01, 04, 05, 12

정상 수신 · 타이밍

START 20ms · WAIT 30us · STOP 60us, 응답 지연, 버튼 재입력, 긴 펄스

28 PASS / 0 FAIL

CASE C TC02, 11, 13, 16

비트 판정 · 데이터

교대 비트, 판정 여유 (45/55us), 바이트 순서, checksum 넘침

27 PASS / 0 FAIL

CASE D TC03, 14

Checksum 오류

valid = 0, data 유지, done 은 출력

10 PASS / 0 FAIL

CASE E TC06 ~ 09

Timeout

SYNC_L/H · DATA_SYNC · BIT 타임아웃 후 IDLE 복귀

30 PASS / 0 FAIL

CASE F TC15

연속 측정

done 직후 재시작, 이전 data 유지 후 교체

7 PASS / 0 FAIL

CASE G 공통 모니터

FSM · 출력 규칙

전이 T1~T13, done 1clk, 출력 규칙 M4~M7

22 PASS / 0 FAIL

TOTAL

138 PASS

0 FAIL · 100 %

※ 알려진 한계(L3) : 타임아웃·리셋 직후 FPGA 가 선을 HIGH 로 구동하는 동안 센서 LOW 와 겹치는 구간 (TC07·09·10, 약 0.3~1 ms) 은 FAIL 이 아닌 기록 항목

DHT 검증 파형 : 시나리오 02 (교대 비트)

| TC02 — 85.0 % / 42.5 ℃, data = 55_00_2A_05_84

선택 이유 | 0 / 1 비트가 계속 교대로 발생 → **비트 판정·시프트 오류가 생기기 쉬운 환경**

② **전체 구간 : WAIT → done (44.951 ~ 48.759 ms, 40 bit 수신)**

INPUT / STIMULUS

센서 모델이 55_00_2A_05_84 전송
0x55 · 0x2A : 0 / 1 이 교대 (27 ↔ 70us)

EXPECTED

수신 중 : data_reg 시프트, data 유지, valid = 0
완료 시 : done 1clk, data 갱신, valid = 1

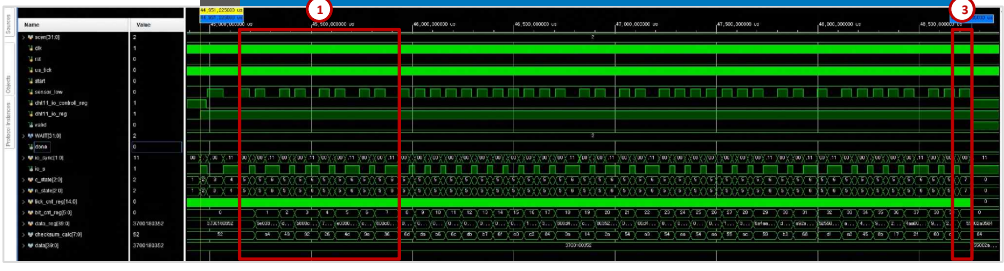
OBSERVED / CHECKED

수신 중 valid = 0, data = 이전 프레임 유지
끝에서 data = 55002a0584, valid = 1 · 7 PASS / 0 FAIL

WAIT · SYNC

DATA 40 bit 수신 (DATA_SYNC ↔ BIT 반복, bit_cnt_reg 0 → 39)

STOP → done



DHT 검증 파형 : 시나리오 02 확대 ①

TC02 — 첫 바이트(0x55) 8 bit : HIGH 폭으로 0 / 1 을 구분

① **첫 바이트 8 bit 확대 (45.146 ~ 45.934 ms) → 0 1 0 1 0 1 0 1 = 0x55**

dht11_io : HIGH 폭 = 비트 값

- 27us → 0, 70us → 1 이 번갈아 나옴
- 0x55 = 0101 0101

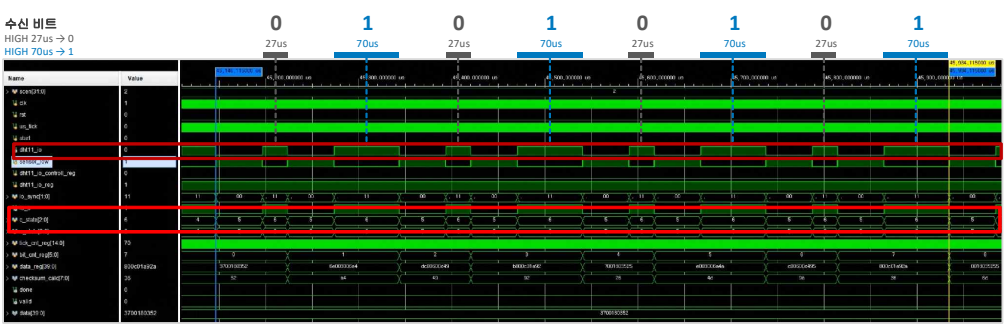
FSM : 비트마다 반복

- c_state 5 (DATA_SYNC) ↔ 6 (BIT)
- bit_cnt_reg 0 → 8

데이터 : 시프트만 진행

- data_reg 가 1 bit 씩 시프트
- data 는 이전 값 유지, valid = 0

수신 비트
HIGH 27us → 0
HIGH 70us → 1



DHT 검증 파형 : 시나리오 02 확대 ③

③ STOP → done 확대 (48.699 ~ 48.77 ms)

Ⓐ STOP 진입 (48.699 ms)

- c_state 6 → 7 (STOP), 센서가 선을 LOW 로 50us 유지
- bit_cnt_reg 39 → 0
- data_reg = 55002a0584 (40 bit 완성)

Ⓑ 센서 해제 (48.749 ms)

- 50us 후 센서가 선을 놓음 : dht11_io · io_s 0 → 1
- io_sync 00 → 11 (2-FF 동기화), FPGA 는 STOP 유지

Ⓒ done (48.759 ms)

- STOP 60us 완료 → IDLE (c_state 0), done 1clk
- valid 0 → 1
- dht11_io_controll_reg 0 → 1

cf) dht11_io_controll_reg : 0 -> 센서 / 1 -> FPGA



HC-SR04 초음파 센서 개요 및 동작 원리



FPGA ↔ HC-SR04 신호 연결



TRIG: 측정 시작 요청 | ECHO: 왕복 시간에 대응하는 펄스

센서 주요 특징

- 약 40kHz 초음파를 이용한 비접촉 거리 측정
- TRIG/ECHO 디지털 신호 기반 제어

거리 측정 과정

TRIG 입력 › 초음파 발사 › 반사파 수신 › ECHO 시간 측정

핵심 원리 | ECHO HIGH 시간을 측정하고 이를 거리 값으로 변환

TRIG/ECHO 타이밍 및 거리 계산

100MHz Clock 기반 TRIG 생성과 ECHO HIGH 시간 측정

① TRIG / ECHO 신호 타이밍

Y축: 디지털 논리 레벨



동작 순서를 보여주는 개념 파형

② TRIG 생성

100MHz = 10ns/Clock
1100 Clock × 10ns = 11µs

③ 거리 계산

거리(cm) ≈ ECHO HIGH(µs) / 58

④ Testbench 입력 예시

600µs → 10cm
1200µs → 20cm

HC-SR04 FSM 상태전이 및 제어 과정

5-State FSM 기반 TRIG 생성, ECHO 감지 및 Timeout 처리

① FSM 상태전이도



Timeout: WAIT 또는 DISTANCE 상태에서 각 상태의 30ms 제한을 초과하면 STOP으로 전이

② 상태별 주요 동작

IDLE	시작 신호 대기 / 카운터 초기화	**START**	TRIG HIGH 11μs 생성	**WAIT**	ECHO 상승 에지 대기
DISTANCE	ECHO HIGH 구간 계수	**STOP**	완료 후 10ms 대기		

HC-SR04 RTL 설계 및 핵심 코드

Controller와 Datapath를 분리하고 1μs Tick을 공통 시간 기준으로 사용

① RTL 모듈 구성

us_tick_gen	100MHz Clock → 1μs Tick
sr04_controller	FSM / TRIG / ECHO / Timeout
sr04 datapath	ECHO 계수 / 거리 계산 / 저장
sr04	Controller + Datapath 통합

1μs Tick 생성 | us_tick_gen.v

```
if (count == F_COUNT - 1) begin
    count <= 0;
    us_tick <= 1'b1;
end else begin
    count <= count + 1'b1;
    us_tick <= 1'b0;
end
```

100MHz Clock을 기준으로 1μs 간격의 Tick을 생성했다.

② 핵심 RTL 코드

측정 완료 및 Timeout | sr04_controller.v

```
assign measure_done =
    (state_reg == DISTANCE) && echo_fall;
assign measure_timeout =
    ((state_reg == WAIT) || (state_reg == DISTANCE)) &&
    (time_cnt >= ECHO_TIMEOUT) && !measure_done;
```

FSM 상태와 ECHO 하강 에지를 이용해 측정 완료를 판정하고, 30ms Timeout을 처리했다.

거리 계산 및 저장 | sr04_datapath.v

```
assign distance_calc = echo_cnt / 16'd58;
always @(posedge clk or posedge rst) begin
    if (rst) distance <= 16'd0;
    else if (done) distance <= distance_calc;
end
```

ECHO 시간 카운트 값을 58로 나누어 거리를 계산하고, done 발생 시 결과를 저장했다.

HC-SR04 시뮬레이션 파형 분석

Vivado Behavioral Simulation으로 정상 거리 측정과 Timeout 동작을 확인

① 정상 거리 측정

TRIG 11μs | ECHO 600μs / 1200μs

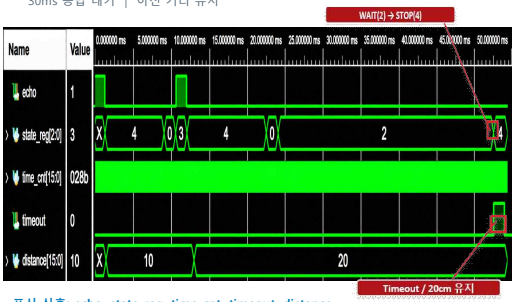


표시 신호: sr04_start, trig, echo, done, distance

검증 항목	확인 내용
펄스 폭	TRIG HIGH = 11μs, ECHO HIGH = 580μs / 1160μs
거리 데이터 갱신	done 발생 시 distance가 0 → 10 → 20으로 갱신
측정 결과	580μs → 10cm, 1160μs → 20cm

② ECHO Timeout

30ms 응답 대기 | 이전 거리 유지



표시 신호: echo, state_reg, time_cnt, timeout, distance

검증 항목	확인 내용
상태전이	ECHO 미입력 시 WAIT(2)에서 30ms 경과 후 STOP(4)으로 전이
timeout	WAIT 상태에서 30ms 경과 후 timeout 펄스 발생
거리 유지	Timeout 발생 후에도 distance가 이전 값인 20cm 유지

HC-SR04 Testbench 검증 전략 및 시나리오

단독 sr04 및 top_sr04를 대상으로 입력 생성 → 출력 비교 → PASS/FAIL 자동 판정

① 검증 구조

입력 생성
100MHz Clock · Reset
TRIG 요청 / ECHO 펄스

→

출력 확인
TRIG · 거리 데이터
Valid · FND · Timeout

→

자동 판정
기대값과 실제값 비교
PASS / FAIL 누적

② 주요 검증 시나리오

검증 대상	입력 조건	확인 항목
Reset / TRIG	초기화 및 측정 시작	초기 출력, TRIG HIGH 11μs
거리 측정	ECHO 580 / 1160μs (sr04)	거리 10cm / 20cm 출력
Top 거리 측정	ECHO 600 / 1200μs (top_sr04)	24-bit 거리 데이터 · Valid 펄스 및 발생 횟수
FND 표시	10cm / 20cm 측정 결과	4자리 스캔 · 숫자 패턴 비교
Timeout	ECHO 응답 미입력	Valid 미발생 · 이전 거리 유지

검증 방식 | Testbench의 task와 check 구문으로 조건을 확인하고 결과를 자동 집계

HC-SR04 검증 결과 및 고찰

단독 센서 처리 로직부터 상위 모듈의 데이터·FND 출력까지 단계별 검증

① 자동 검증 결과

sr04 단독 모듈

7 / 7 PASS | FAIL 0

TRIG 11µs · 10/20cm · Timeout

top_sr04 상위 모듈

18 / 18 PASS | FAIL 0

Valid · 24-bit 데이터 · FND · Timeout

② 실제 검증 결과 요약

검증 단계	확인된 기능
단독 모듈	Reset, TRIG 11µs (3회), 10cm·20cm 거리 측정, Timeout 검출
상위 모듈	TRIG, Valid 펄스 HIGH/LOW 및 횟수, 24-bit 거리 데이터, FND 표시
예외 처리	ECHO 무응답 시 Valid 미발생 및 마지막 거리 20cm 유지

③ 결론 및 고찰

Controller와 Datapath 기반 거리 측정 및 Timeout 처리를 검증했으며, 상위 모듈에서 데이터 전달, Valid 신호 및 FND 출력까지 정상 동작을 확인했다.

UART–FIFO–ASCII

① UART_FIFO_ASCII



② Binary → ASCII 변환 방식 비교

비교 항목	나눗셈	쉬프트 + Add	변화
변환 모듈 LUT	139	76	−63개 (−45.3%)
변환 모듈 Register	61	63	+2개 (+3.3%)
Top(uart-fifo-ascii) LUT	322	251	−71개 (−22.0%)
Top(uart-fifo-ascii) Register	227	229	+2개 (+0.9%)
변환 시간	8클럭 / 80 ns	24클럭 / 240 ns	3배의 시간

쉬프트 + Add: 변환기 LUT 45.3% 절감, 변환 시간은 3배

100 MHz, start 수락 후 done까지.UART 송신 시간 제외

나눗셈 (/10, %10)

Name	Slice LUTs (20800)	Slice Registers (41600)	Bonded IOB (106)	BUFGCTRL (32)
N UART_FIFO_ASCII	322	227	58	1
U_ASCII_SENDER (ascii_sender)	230	163	0	0
[redacted]	130	61	0	0
U_UART_FIFO_LOOPBACK (uart_fifo_loopback)	92	64	0	0

쉬프트 + Add

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Bonded IOB (106)	BUFGCTRL (32)
N UART_FIFO_ASCII	251	229	4	58	1
U_ASCII_SENDER (ascii_sender)	160	165	4	0	0
[redacted]	70	63	0	0	0
U_UART_FIFO_LOOPBACK (uart_fifo_loopback)	91	64	0	0	0

Binary → BCD → ASCII 변환 원리

숫자를 PC에 표시하기 위해, 숫자의 자릿수를 분리하고 문자 코드로 변환

24-bit Binary → 32-bit BCD (8자리 × 4-bit) → 64-bit ASCII (8문자 × 8-bit)

① MSB부터 한 비트씩 시프트 : 기존 값 × 2 + 다음 비트

입력 25 = 0000 0000 0000 0000 00**01 1001** / 19개 비트는 0이므로, 이후 5개 비트를 처리하면서 값이 변화



② 23번째 비트 처리: BCD 각 자리가 5 이상이면 +3 보정



- BCD는 10진수 한 자리를 4-bit로 표현, 각 자릿수에는 **0~9만 저장 가능**
- 시프트 시 값이 2배로 증가하므로, **5 이상인 자릿수에 미리 3을 더해 올바른 자리올림 유도**

③ BCD 25: 0010 0101 → 각 자리 + 0x30 → 문자 "2"=0x32, "5"=0x35

Start 수신 → 24클럭 비트 처리 → "00000025" 출력, done 신호 발생

시뮬레이션 : 주요 검증 시나리오

목적 **명령 수신과 시계 ASCII 송신, FIFO Full 대기·재개 확인**

① 명령 디코딩 · ② 시계 12:00:00:00 송신 · ③ Full에서 문자 유지·재개

검증	검증 상황 · 입력 순서	예상 결과값 · 확인 내용	선정 이유
UART → FIFO RX → ASCII Decoder	UART '0', '1', '2', '3' 순서로 입력 이후 'r', 'R'을 연속 입력 각 문자 수신 후 rx_done 발생	0/1/2/3: 해당 모드 요청 출력 stopwatch / watch / sr04_start / dht11_start 순서로 1클럭 펄스 r/R: 모두 uart_R 출력	수신한 명령과 제어 출력의 대응 관계 확인
가상 시계 데이터 → 숫자 변환 → FIFO TX → UART	UART '1', result_type=01 시계 12:00:00:00 결과 입력 result_data=0x00600000 result_valid=1에서 result_done을 1클럭(10 ns) 인가	시계: 12:00:00:00 + CR/LF 13바이트 순차 송신 숫자 12 → ASCII 00000012 필드별 24클럭(240 ns) 변환	시계 값의 숫자 변환과 출력 서식, UART 송신 결과를 순서대로 확인
시계 송신 중 FIFO TX Full 대기·재개	시계 문자열을 FIFO에 연속 저장 FIFO Full이 되면 저장 대기 UART가 문자를 가져가 공간 확보 대기하던 문자를 저장하고 재개	Full 중 push=0, char_index=5 다음 저장 문자 ":" 유지 공간 확보 후 push 재개 ":" 저장 시 char_index 5 → 6	FIFO가 가득 찬 동안 문자와 순서를 유지하고 저장을 재개하는지 확인

Uart rx -> Ascii decoder

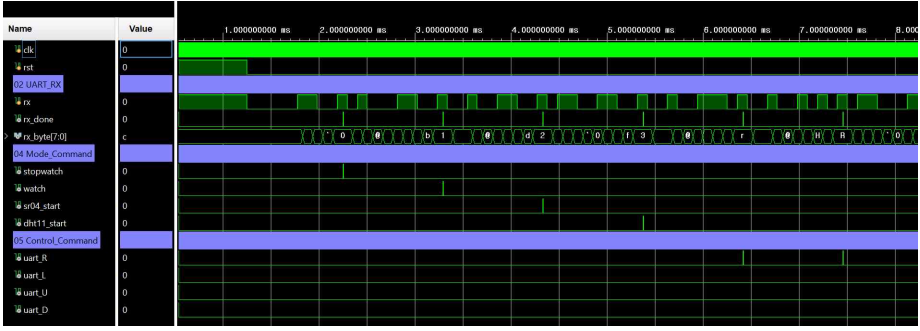
모든 동작의 시작점인 명령 전달을 확인

예상

- ASCII 명령 0, 1, 2, 3 수신 시 각 모드의 제어 신호가 출력
- r/R 수신 시 uart_R=1 출력

결론

rx_done 발생 이후 수신 명령에 대응하는 stopwatch, watch, sr04_start, dht11_start 펄스 확인



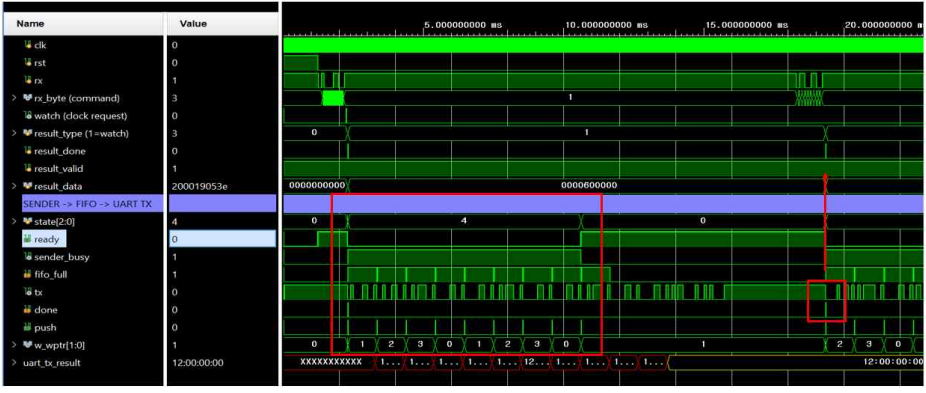
Ascii sender -> uart

예상

- 시계 모드(result_type=1)에서 초기값 12:00:00:00을 ASCII로 변환하여 순차 전송
- SEND 상태에서 FIFO에 공간이 있을 때 push=1 발생

State = 4 : data send
Push : (state == send) && ~fifo_full

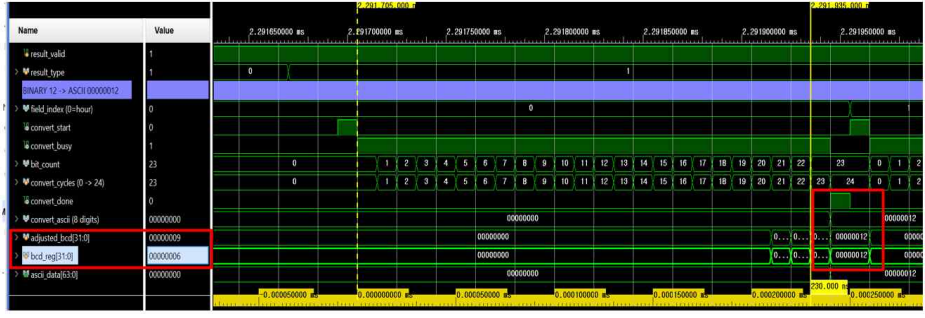
결론 **ASCII 문자열의 FIFO 저장 및 UART 순차 전송을 통한 시계 데이터 출력 확인**



Binary -> Ascii

예상 시계 Binary 데이터 0x00600000(12시 00분 00초 00)을 ASCII 문자열 "12" 변환

결론 **uart_tx_result에서 "12:00:00:00" 출력 확인**



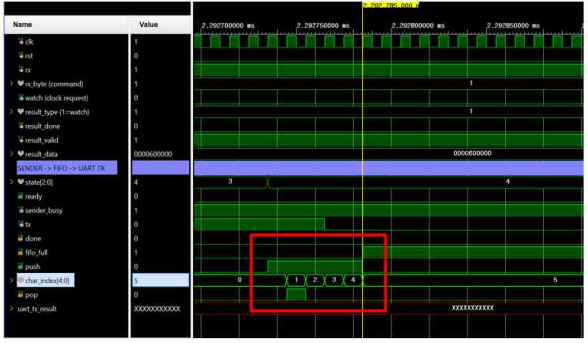
FIFO TX -FULL

예상

- FIFO Full 발생 시 push=0 및 char_index 유지
- Pop으로 공간 확보 후 Push 재개 및 char_index 증가

결론

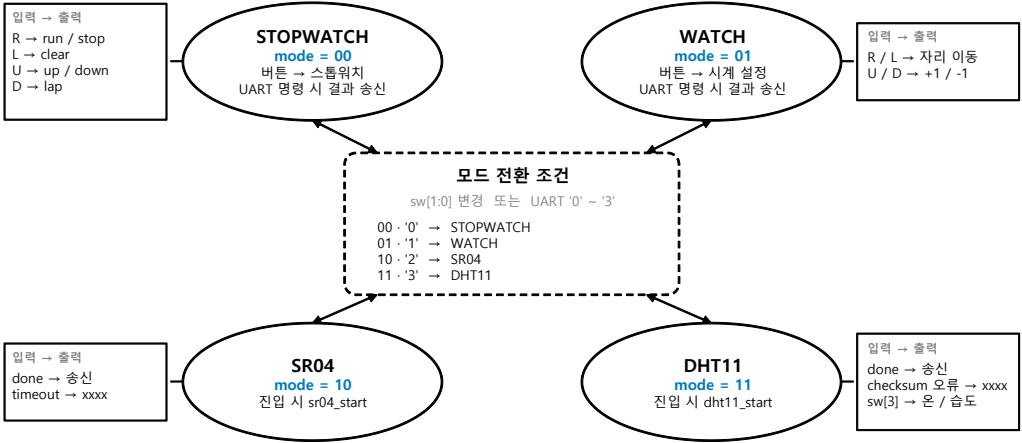
FIFO Full 발생 시 미저장 문자 대기 및 공간 확보 후 저장 재개 동작 확인





컨트롤 유닛 : 동작 흐름

결론 어느 모드에서든 sw 변경 · UART '0'~'3' 으로 모드 전환, 버튼 등의 입력은 현재 모드 모듈에만 전달



원 안 = 상태가 정하는 출력 (display_mode) · 박스 = 현재 상태 + 입력으로 정해지는 출력 · 송신 요청은 저장 후 sender_ready 일 때 1회

통합 시뮬레이션 : 시나리오

결론 **통합 시뮬레이션 파형 ① ~ ④ 의 기본 동작 확인 항목 모두 Expectation 과 일치 (4 / 4 PASS)**

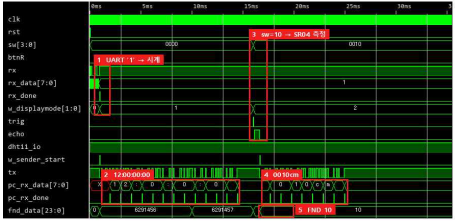
No	파형 (사진 제목)	Input Stimulus	Expectation	결과
①	모드 전환 시계 (UART) → SR04 (스위치)	UART '1' → 시계	12:00:00:00 송신	PASS
		sw = 10 → SR04	0010cm 송신	PASS
		FND 거리	10	PASS
②	모드 전환 DHT11 (스위치)	sw = 11 → DHT11	start 20 ms → 40 bit	PASS
		측정 완료 → 송신	H=045.00% T=025.30C	PASS
		FND 온도	2530	PASS
③	모드별 버튼 분배	시계 모드 btnR	스톱워치 미동작	PASS
		UART '0'	00:00:00:00	PASS
④	스톱워치 동작 버튼 run → UART stop	btnR → run	FND 0 → 5	PASS
		UART 'r' → stop	FND 5 유지	PASS
		UART '0'	00:00:00:05 송신	PASS

TB 모델 : PC 9600 baud · SR04 echo 580 us = 10 cm · DHT11 습도 45.0 % / 온도 25.3 ℃ | ① ② ④ : tb_project_top · ③ : tb_system_exception

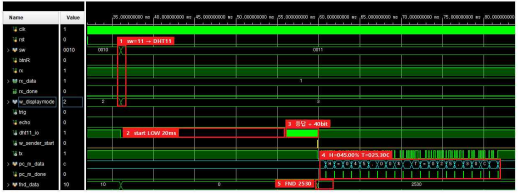
통합 시뮬레이션 : 모드 전환 · 센서 측정

결론 **UART · 스위치 모드 전환 후 해당 모드 값 송신, 센서는 측정 완료 시 자동 송신**

① 모드 전환 : 시계 (UART) → SR04 (스위치)



② 모드 전환 : DHT11 (스위치)



Expectation	파형 결과
UART '1' → 시계	12:00:00:00 송신
sw = 10 → SR04	0010cm 송신
FND 거리	10

Expectation	파형 결과
sw = 11 → DHT11	start 20 ms → 40 bit
측정 완료 → 송신	H=045.00% T=025.30C
FND 온도	2530

통합 시뮬레이션 : 버튼 분배 · 스톱워치

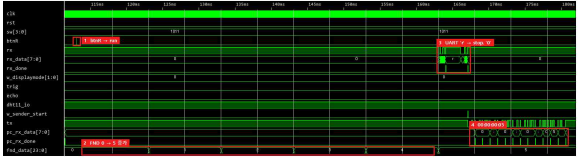
결론 버튼은 현재 모드 모듈에만 전달, 버튼과 UART 명령이 같은 기능으로 동작

③ 모드별 버튼 분배



Expectation	파형 결과
시계 모드 btnR	스톱워치 미동작
UART '0'	00:00:00:00

④ 스톱워치 동작 : 버튼 run → UART stop

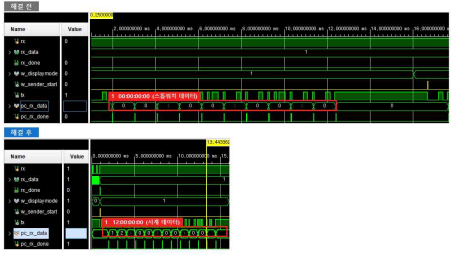


Expectation	파형 결과
btnR → run	FND 0 → 5
UART 'r' → stop	FND 5 유지
UART '0'	00:00:00:05 송신

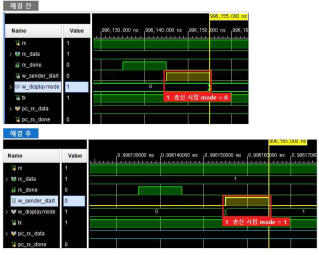
트러블슈팅 ① : 모드 전환 직후 송신

결론 **송신 요청을 래치해 전환 다음 클록에 송신 → 00:00:00:00 에서 12:00:00:00 으로 정상화**

① 모드 전환 직후 이전 모드 데이터 송신



① 확대 : 송신 시점의 w_displaymode

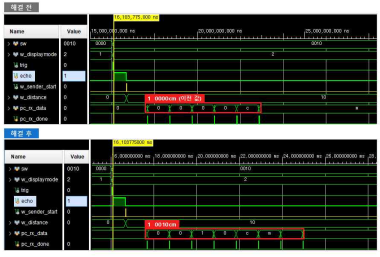


단계	내용
① **현상**	UART '1' 후 이전 모드 값 송신
② **원인**	전환과 같은 클록에 송신 (mode = 0)
③ **해결**	다음 클록 송신

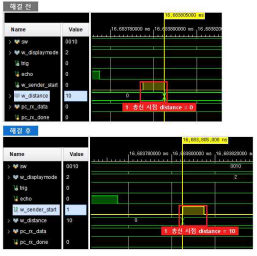
트러블슈팅 ② : SR04 이전 값 송신

결론 **①의 래치로 distance 갱신 뒤 송신 → 0000cm 에서 0010cm 로 정상화**

② SR04 측정 후 이전 거리 값 송신



② 확대 : 송신 시점의 w_distance

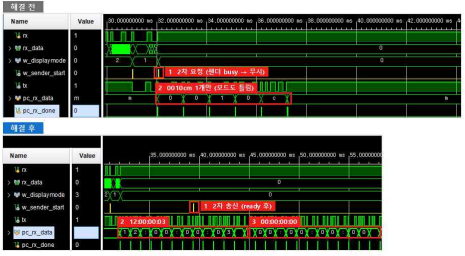


단계	내용
① **현상**	10 cm 인데 0000cm 송신
② **원인**	distance 가 done 다음 클록에 갱신
③ **해결**	다음 클록 송신
④ **결과**	**0000cm → 0010cm**

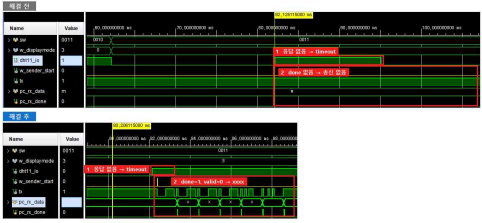
트러블슈팅 ③ ④ : 예외 처리

결론 샌더 busy 중 요청을 유지, 센서 무응답에도 done 출력 → 모든 명령에 응답

③ 연속 명령 시 응답 누락



④ 센서 무응답 시 송신 없음



단계	내용
① **현상**	'1', '0' 연속 입력 → 응답 1개
② **원인**	샌더 busy 중 들어온 요청 무시
③ **해결**	sender_ready 까지 요청 유지
④ **결과**	**응답 1개 → 응답 2개**

단계	내용
① **현상**	DHT11 무응답 → PC 에 송신 없음
② **원인**	timeout 때 done 이 없음
③ **해결**	timeout 에도 done, valid = 0
④ **결과**	**송신 없음 → xxxx**

결론 : 설계 · 검증 결과

결론 **4개 모드 통합 동작 시뮬레이션 4 / 4 PASS, 트러블 4 / 4 해결, Basys-3 보드 정상 동작 확인**

4 / 4	4 / 4	4 모드
통합 시나리오 PASS	트러블 해결 (전 · 후 비교)	Basys-3 보드 동작 확인

기능	검증 내용 (수치)	시뮬레이션	보드
스톱워치	btnR run · UART 'r' stop · '0' → 00:00:00:05 송신	PASS	확인
시계	UART '1' 전환 · 12:00:00:00 송신	PASS	확인
SR04	echo 580 us → 10 cm (0010cm 송신, FND 10)	PASS	확인
DHT11	습도 45.0 % · 온도 25.3 ℃ 송신, FND 2530 / 4500 전환	PASS	확인
예외 처리	무응답 · checksum 오류 → xxxx, 연속 명령 2개 모두 응답	PASS	-

고찰 : 향후 계획

결론 신호 규약 · 보드 예외 확인 · PC 출력 구분 · 자동 재측정으로 완성도를 높일 계획

No	항목	현재	향후 계획
1	신호 규약	done · valid · ready 등의 신호 규약을 통합 단계에서 팀원들과 함께 맞춤	**설계 초기에** **팀 공통 신호 규약표부터 작성**
2	보드 예외 확인	센서 무응답 · checksum 오류는 시뮬레이션으로만 확인	**센서 연결을 분리하는 등** **보드에서도 예외 동작 확인**
3	PC 출력 구분	스톱워치 · 시계 송신 형식이 같아 (00:00:00:00) 어느 모드 값인지 구분 불가	**출력 앞에 SW / WT 표시를 붙여** **모드 구분**
4	측정 실패 처리	센서 무응답 시 바로 xxxx 송신	**1회 자동 재측정 후에도** **실패하면 xxxx 송신**